



南開大學

从线程到协程

高并发 I/O 场景下轻量级并发机制的底层实现与实验分析

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

2026 年 6 月 1 日

目录

1 引言	3
1.1 研究背景与问题提出	3
1.2 研究意义	3
1.3 本文的研究内容与结构安排	3
2 并发模型演进与协程产生的动因	3
2.1 从进程到线程	3
2.2 线程模型在高并发 I/O 场景下面临的瓶颈	4
2.3 事件驱动与异步回调模型的优缺点	4
2.4 协程产生的核心动因	4
2.5 本章小结	4
3 协程的核心思想与基本机制	5
3.1 协程的定义与基本特征	5
3.2 协程与函数、线程、进程的区别	5
3.3 挂起、恢复与协作式调度	5
3.4 有栈协程与无栈协程	6
3.5 协程的典型适用场景	6
3.6 本章小结	7
4 协程的底层实现机制	7
4.1 协程的执行流程概述	7
4.2 编译器视角：状态机转换与协程帧	7
4.3 运行时视角：调度器与任务管理	7
4.4 操作系统视角：线程调度与 I/O 事件通知	8
4.5 编译器、运行时与操作系统的协同关系	8
4.6 本章小结	9
5 典型编程语言中的协程支持与工程实践	9
5.1 C++20 协程机制	9
5.2 Go 语言中的 goroutine	9
5.3 Python 中的 <code>asyncio</code> 与 <code>async/await</code>	10
5.4 Java 虚拟线程的轻量级并发机制	10
5.5 不同语言协程支持方式的对比分析	10
5.6 协程在工程实践中的应用与边界	11
5.7 本章小结	11
6 实验设计与结果分析	11
6.1 实验目标	11
6.2 实验环境与评价指标	11
6.3 实验一：模拟 I/O 密集型任务下协程与多线程对比	12
6.4 实验二：真实网络 I/O 场景下协程与多线程对比	12
6.5 实验三：CPU 密集型任务下协程与多线程对比	13
6.6 实验四：Python 高并发扩展实验	13
6.7 实验五：Java 21 平台线程与虚拟线程扩展实验	13
6.8 基础机制验证实验结果	14
6.9 Python 高并发扩展实验结果	15
6.10 Java 21 平台线程与虚拟线程扩展实验结果	18
6.11 综合讨论	20
6.12 本章小结	20

7 结论与展望	20
7.1 全文总结	20
7.2 主要结论	20
7.3 未来发展趋势与进一步研究方向	21
A 附录：实验代码	21
A.1 GitHub 项目链接	21
A.2 多线程实验代码	21
A.3 协程与虚拟线程实验代码	22
A.4 数据处理与绘图代码	22
B 附录：AI 辅助使用说明	22
B.1 AI 辅助内容	22
B.2 人工核验与修改说明	22

1 引言

1.1 研究背景与问题提出

随着网络服务、云计算、微服务、在线推理系统和云原生基础设施的并发度不断提升，程序设计者越来越频繁地面对这样一类问题：单个任务的计算量并不大，但任务数量很多，并且大量时间消耗在网络、磁盘、数据库或 RPC 等 I/O 等待上。若仍采用传统的“一请求一线程”思路，虽然编程模型直观，但线程创建、栈空间占用、上下文切换和阻塞等待都会迅速累积，导致资源利用率下降。

Python 官方文档指出，`asyncio` 是一个利用 `async/await` 语法编写并发代码的异步 I/O 库，尤其适合 I/O 绑定和高层结构化网络代码 [1,2]。OpenJDK 的 JEP 444 也明确给出了相似判断：虚拟线程面向的是高吞吐并发应用，用更轻量的执行实体降低编写和维护并发程序的难度 [3]。Go 官方文档则从另一个角度说明了这一点：`goroutine` 是轻量级并发执行单元，它们会被运行时复用到一组操作系统线程之上 [4-6]。这些事实共同说明，现代并发编程的发展重点，已经从“如何创建更多线程”，转向“如何用更低成本组织大量等待中的任务”。

因此，一个自然的问题是：并发执行机制为什么会从进程、线程进一步演进到协程式并发模型？本文围绕这个问题，从概念、底层机制、语言支持以及实验结果四个层面给出系统回答。

1.2 研究意义

本报告的意义主要体现在以下三个方面。

第一，从知识结构上看，协程是现代系统与应用编程中的关键主题。它不仅出现在 Python 的 `asyncio`、Go 的 `goroutine`、Java 21 的虚拟线程中，也在 C++20 中进入了标准语言层。理解协程，有助于把看似分散的语言特性统一到“轻量级并发抽象”这一框架下。

第二，从工程实践上看，协程并不是“线程的简单替代品”。它在 I/O 密集型场景中往往优势显著，但在 CPU 密集型场景中并不会天然加速计算。因此，对协程的价值与边界进行辨析，比单纯强调“协程更快”更有意义。

第三，从课程训练上看，OT-协程技术调研不仅要求对概念与原理进行综述，还要求给出实验、比较和个人判断。这意味着报告必须把概念、底层机制、语言实现和定量实验组织成一条完整主线，而不是零散堆砌资料。

1.3 本文的研究内容与结构安排

围绕上述问题，本文的内容安排如下：第二章讨论并发模型的演进路径，解释协程产生的动因；第三章给出协程的定义、特征、分类与适用场景；第四章从编译器、运行时和操作系统三个层面分析协程的底层实现；第五章比较 C++20、Go、Python 和 Java 中的协程式并发支持；第六章给出本文的实验设计与结果分析，其中既包括基础机制验证实验，也包括高并发扩展实验；第七章总结全文并讨论未来趋势。

2 并发模型演进与协程产生的动因

2.1 从进程到线程

早期并发程序主要依赖进程。进程隔离性强、故障边界清晰，但创建与切换成本较高，进程间通信也相对笨重。线程的出现部分解决了这一问题：同一进程内的线程共享地址空间，使得数据共

享更方便、创建成本更低，因此线程很快成为主流并发机制。

然而，线程模型并没有消除并发程序的全部复杂性。随着任务数量增加，线程本身也变成需要被管理和调度的资源。特别是在高并发 I/O 场景中，大量线程虽然“存在”，却并不真正执行计算，而只是处于阻塞等待状态，这种状态会浪费栈空间和调度开销。

2.2 线程模型在高并发 I/O 场景下面临的瓶颈

在高并发 I/O 场景中，线程模型面临的主要瓶颈包括以下几点。

- (1) **内存开销**：每个线程通常都有独立栈空间，当线程数达到数千时，内存占用会迅速增长。
- (2) **切换成本**：线程调度由内核参与，频繁切换会带来额外上下文切换成本。
- (3) **阻塞传播**：当线程因 I/O 阻塞时，该线程上的执行能力无法继续利用。
- (4) **调优复杂度**：为限制线程数量，工程上常采用线程池；但线程池大小、阻塞策略和排队行为都需要额外调优。

这些问题并不意味着线程“过时”，而是说明线程更适合作为操作系统层面的并行执行资源，而不一定适合作为海量等待型任务的一一映射单位。

2.3 事件驱动与异步回调模型的优缺点

为了解决“一请求一线程”带来的资源浪费，工程中逐渐发展出事件驱动与异步回调模型。其核心思想是：通过事件循环和 I/O 多路复用，让少量线程处理大量连接；当 I/O 未就绪时，不阻塞整个执行流，而是注册回调，在事件到来后再继续处理。

这种模型在资源利用率上有明显优势，但它也带来了新的问题：程序控制流被拆成多个回调片段，局部变量与错误处理逻辑不再自然地写在同一个函数中，容易形成所谓“回调地狱”。因此，事件驱动模型在效率上解决了问题，却在可读性和维护性上引入了新的负担。

2.4 协程产生的核心动因

协程式并发的核心目标，可以概括为一句话：**在保留异步 I/O 高效率的同时，尽量恢复同步代码的书写体验。**

Python 的 `asyncio` 文档表明，协程与任务是其高层 API 的核心组成部分，程序员可以用接近顺序代码的方式组织异步逻辑 [1,2]。Go 官方 FAQ 说明，goroutine 的设计思想就是将独立执行的函数复用到线程集合之上，使阻塞不会自动“拖死”同一线程上的其他并发任务 [5]。OpenJDK 的虚拟线程则进一步说明，即使保留“阻塞式写法”，也可以通过更轻量的线程实现高吞吐并发应用 [3]。

因此，协程并不是从“功能”上取代线程，而是从**抽象层次**上重新组织等待型任务：把“任务”的语义交给语言或运行时，把“真正并行执行”的语义留给底层线程和操作系统。

2.5 本章小结

本章说明了并发模型从进程到线程、再到事件驱动与协程式并发的演进逻辑。线程解决了进程过重的问题，但在高并发 I/O 场景中仍存在明显资源浪费；事件驱动模型提高了效率，却降低了代码直观性；协程的出现，正是为了在效率与可维护性之间取得更好的平衡。

3 协程的核心思想与基本机制

3.1 协程的定义与基本特征

从抽象上看，协程可以理解为可在执行过程中主动挂起、并在之后从挂起点恢复执行的函数或任务。与普通函数“调用—执行—返回”的单一控制流不同，协程允许把一次逻辑执行拆成多个阶段，每次恢复时都从上次暂停的位置继续。

协程通常具有以下基本特征。

- (1) **可挂起与可恢复**：程序可以在等待某个事件时暂停，而不是阻塞整个线程。
- (2) **轻量级**：协程状态通常只保存必要的执行现场，而不是为每个任务分配完整线程资源。
- (3) **用户态调度**：大量协程的调度往往由运行时在用户态完成，而非完全依赖内核。
- (4) **更自然的控制流**：协程代码通常比显式回调更容易理解和维护。

3.2 协程与函数、线程、进程的区别

为了避免概念混淆，表 1 总结了协程与函数、线程、进程的主要区别。

表 1: 函数、协程、线程与进程的比较

对象	调度主体	是否可中途挂起	资源开销	典型作用
函数	调用者	否	很低	封装顺序逻辑
协程	运行时/事件循环	是	低	表达大量等待型任务
线程	操作系统内核	可被抢占	中等到较高	提供并行执行资源
进程	操作系统内核	可被抢占	高	提供隔离与独立地址空间

这里需要特别强调两点。第一，**协程不等于线程**，它通常建立在线程之上；第二，**协程也不等于并行**，单线程中的多个协程可以并发推进，但未必真正同时在多个 CPU 核上执行。

3.3 挂起、恢复与协作式调度

协程最核心的运行机制是挂起（suspend）与恢复（resume）。在等待 I/O、定时器或其他异步结果时，协程会保存当前状态，把控制权交还给调度器；当条件满足后，调度器再恢复该协程继续执行。

这类调度通常是**协作式的**：任务需要在合适的位置显式让出执行权，而不是像线程那样主要依靠内核抢占。在 Python 的 `asyncio` 中，协程通过 `await` 交还控制流给事件循环，任务则由事件循环统一调度 [1,2]。在 Go 运行时中，`gopark` 与 `goready` 等底层机制反映了 `goroutine` 的挂起与唤醒逻辑 [6]。

图 1 给出了一个简化的协程执行流程。

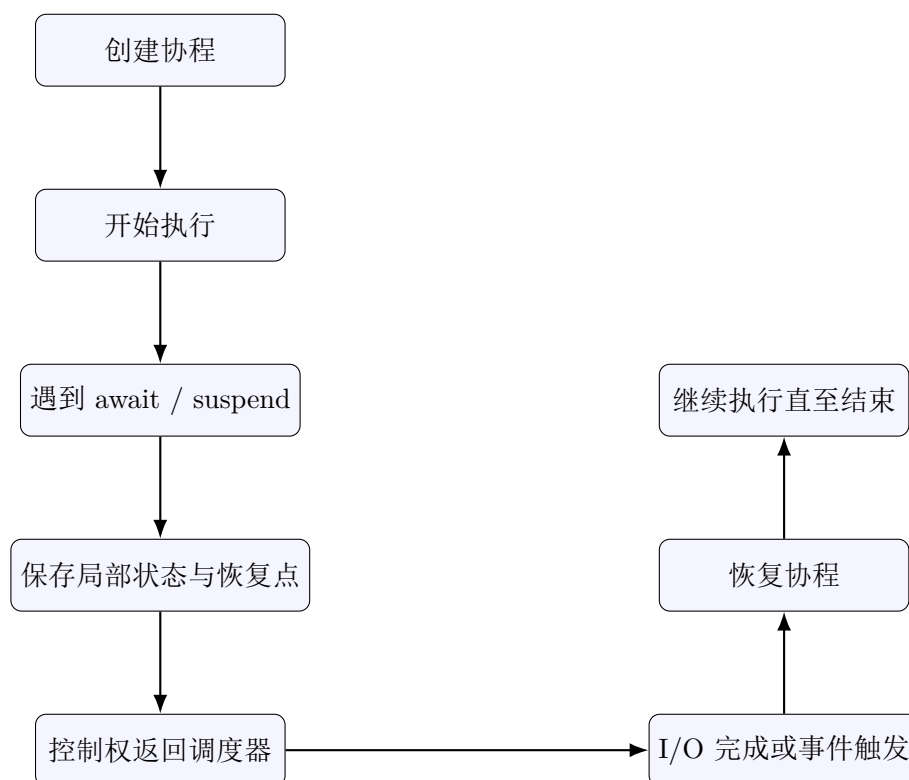


图 1: 协程一次挂起与恢复的简化流程图

3.4 有栈协程与无栈协程

按照状态保存方式，协程可粗略分为有栈协程（`stackful coroutine`）和无栈协程（`stackless coroutine`）。

有栈协程拥有相对独立的调用栈，因此切换时更像“轻量线程”，可在更灵活的位置暂停；代价是调度与栈管理更复杂。无栈协程通常只在语言规定的挂起点保存状态，其恢复点更明确，实现也更便于被编译器转换为状态机。MSVC 文档说明，标准 C++ 协程在 C++20 中已经成为语言级特性，编译器围绕 `co_await`、`co_yield` 和 `co_return` 提供支持 [7]；从实现风格上看，C++20 协程更接近无栈协程。

3.5 协程的典型适用场景

协程特别适合以下场景：

- 高并发网络服务器；
- 爬虫、网关、代理、RPC 框架；
- 数据库连接访问、分布式任务调度；
- 含大量等待时间的请求处理流水线。

与之相对，若任务几乎全部时间都在做数值计算、图像卷积或矩阵运算，那么瓶颈主要来自计算本身，此时多线程、SIMD、GPU 等机制往往比协程更重要。换言之，协程的价值主要体现在降低等待成本，而不是缩短纯计算路径。

3.6 本章小结

本章对协程的概念、特征、调度方式与分类进行了说明。协程的本质不是“更小的线程”，而是对等待型任务的一种更细粒度抽象；它通过挂起与恢复机制，让程序在高并发 I/O 场景下兼顾效率与可读性。

4 协程的底层实现机制

4.1 协程的执行流程概述

从底层看，一个协程至少要回答三个问题：

- (1) 在哪里保存挂起时的状态？
- (2) 挂起之后由谁来决定何时恢复？
- (3) 恢复时如何回到正确的执行位置？

这三个问题分别对应编译器、运行时与操作系统的职责划分。编译器负责把高级语义转换成可执行表示，运行时负责管理协程对象与调度队列，操作系统则继续负责底层线程和 I/O 事件通知。

4.2 编译器视角：状态机转换与协程帧

对现代编译器而言，协程并不是“神秘的黑箱”，而更像一种带有显式挂起点的函数变换。MSVC 文档指出，C++20 标准协程由编译器直接支持，关键字包括 `co_await`、`co_yield` 和 `co_return` [7]。Clang 文档则进一步揭示了其实现细节：前端先生成单个 LLVM IR 函数，再由中端的 `CoroSplit` 等过程将其拆分为 `ramp`、`resume` 和 `destroy` 等函数，同时把跨越挂起点仍然存活的局部变量搬移到协程帧中 [8]。

这意味着，从编译器视角看，协程本质上是一台状态机：

- 每个 `await` 对应一个可能的状态切换点；
- 局部变量若要跨越挂起点继续存在，就必须从普通栈迁移到协程帧；
- 恢复协程时，本质上是跳转到状态机中相应的恢复入口。

4.3 运行时视角：调度器与任务管理

编译器解决的是“如何表示协程”，而运行时解决的是“如何管理大量协程”。不同语言对此的策略并不相同。

在 Python 中，`asyncio` 提供事件循环、任务 (Task)、Future 等高层 API，用来运行和调度协程 [1,2]。在 Go 中，goroutine 由运行时调度到一组操作系统线程上；官方 FAQ 强调，当某个 goroutine 因阻塞系统调用而停滞时，运行时会把其他 goroutine 迁移到可运行线程上，以免整个线程白白等待 [5]。在 Go runtime 的内部实现说明中，还能看到 goroutine 调度与 OS 调度之间存在显式区分，例如 `gopark` 和 `goready` 直接与 goroutine 调度器交互，而不是直接调用操作系统调度器 [6]。

从工程角度看，运行时层的关键工作包括：

- 创建任务对象并维护其生命周期；

- 把“可运行任务”放入队列；
- 在 I/O 完成或定时器到期时唤醒对应任务；
- 在必要时与线程池或 OS 线程协同，实现真正的并行执行。

4.4 操作系统视角：线程调度与 I/O 事件通知

协程通常运行在用户态，但这并不意味着操作系统“不重要”。相反，协程能否高效执行，往往依赖于操作系统提供的底层能力，例如：

- 线程与 CPU 调度；
- 网络套接字与文件 I/O；
- 定时器、中断与事件通知；
- I/O 多路复用机制（如 `epoll`、`kqueue`、`IOCP` 等）。

因此，协程并不是“脱离线程独立存在”的技术，而是一种建立在 OS 线程与事件通知机制之上的上层抽象。协程将海量任务组织在少量线程上运行，但这些线程仍然需要内核提供真正的调度与 I/O 支持。

4.5 编译器、运行时与操作系统的协同关系

把三层放在一起看，可以得到一个更清晰的结论：

协程机制的实现不是某一层单独完成的，而是语言语义、运行时调度与操作系统能力共同作用的结果。

如果没有编译器，协程语法无法被变换成状态机；如果没有运行时，协程对象无法被高效调度；如果没有操作系统，线程、定时器与 I/O 事件也无从而来。也正因为如此，不同语言虽然都提供“轻量级并发”能力，但它们在抽象接口、运行时责任和性能特征上存在明显差异。

图 2 用分层方式总结了编译器、运行时与操作系统之间的关系。

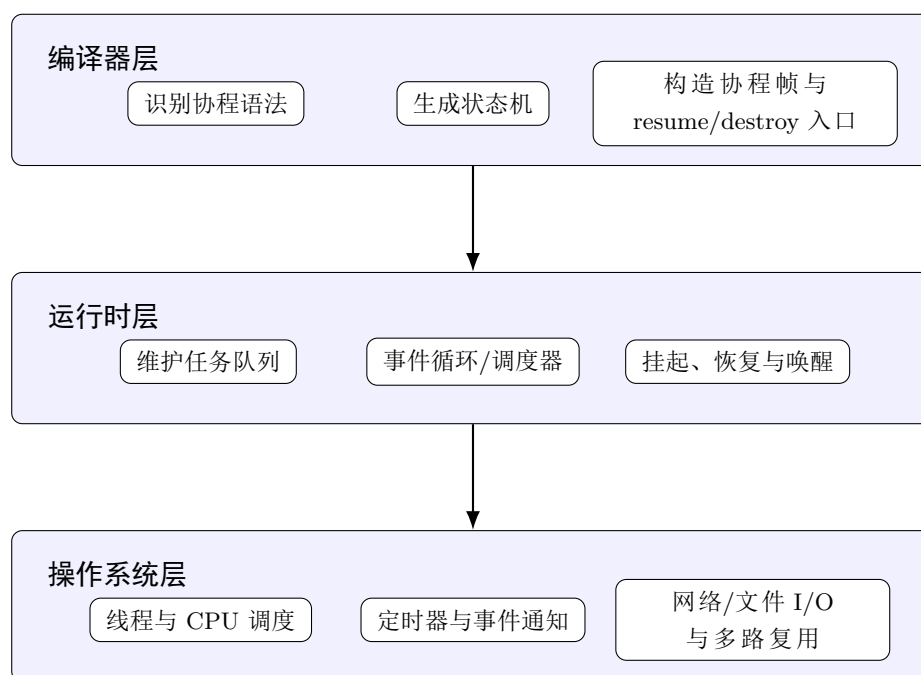


图 2: 协程实现中的编译器、运行时与操作系统分层关系

4.6 本章小结

本章从底层角度分析了协程的实现逻辑。协程不是简单的语法糖，也不是线程的直接替代品，而是一套由编译器、运行时与操作系统共同支撑的并发机制。理解这一点，有助于正确把握后续语言对比与实验分析中的差异来源。

5 典型编程语言中的协程支持与工程实践

5.1 C++20 协程机制

C++20 将协程纳入标准语言本身。MSVC 文档表明，在 C++20 或更高标准下，标准协程已默认可用；相关关键字为 `co_await`、`co_yield` 和 `co_return` [7]。这说明 C++20 更像是把“协程最底层的语言机制”交给编译器，而不是像某些语言那样同时提供完整的高级异步运行时。

因此，C++20 协程的特点是：**语言机制强、标准运行时弱、库设计空间大**。它非常适合做高性能框架、网络库和嵌入式异步基础设施，但使用门槛相对较高。开发者不仅要理解协程语法，还要理解 `promise_type`、`awaiter`、调度器接口以及资源生命周期问题。

5.2 Go 语言中的 goroutine

Go 语言并不把 goroutine 定义为“标准协程”一词下的某个严格变体，但从工程目标上看，它显然属于轻量级并发执行单元。Effective Go 指出，goroutine 是在同一地址空间中并发执行的函数；它开销很低，栈从很小开始并可按需增长；如果某个 goroutine 因 I/O 等原因阻塞，其他 goroutine 仍可继续执行 [4]。FAQ 进一步强调了 goroutine 会被复用到一组线程之上 [5]。

Go 的优势在于语言与运行时的一体化设计：

- 语法上用 `go` 关键字即可启动并发任务；

- 通过 channel 组织 CSP 风格的通信；
- 运行时负责调度与栈管理，隐藏了大量线程管理细节。

这使 Go 在高并发网络服务、网关、中间件和基础设施编程中非常流行。

5.3 Python 中的 `asyncio` 与 `async/await`

Python 的 `asyncio` 是理解协程编程最直接的教材之一。官方文档指出，它可用 `async/await` 编写异步 I/O 代码，尤其适合 I/O-bound 任务 [1]。Coroutines and Tasks 文档则概括了协程、任务、取消和并发运行等高层 API 结构 [2]。

Python 的优势在于抽象层次高、上手快，非常适合快速验证协程与线程在 I/O 场景下的差异。其局限则在于运行时和解释器特性决定了它更擅长 I/O 并发，而不擅长用协程提升 CPU 密集计算速度。因此，本文的实验也选择 Python 作为首个验证平台。

5.4 Java 虚拟线程的轻量级并发机制

JEP 444 给出的定义非常清楚：虚拟线程是轻量级线程，它的目标是显著降低编写、维护和观测高吞吐并发应用的难度，并在 JDK 21 中正式交付 [3]。严格地说，虚拟线程并不等同于语言级协程；但从“以轻量执行单元承载大量等待型任务”的角度看，它与协程式并发具有高度相似的工程目标。

Java 虚拟线程的重要意义在于：它试图在不强迫程序员全面转向回调式异步代码的情况下，保留传统阻塞式写法的可读性，同时获得更好的并发伸缩能力。这为既有 Java 生态向轻量级并发过渡提供了现实路径。

5.5 不同语言协程支持方式的对比分析

表 2 对四种语言的协程式并发支持进行总结。

表 2: 不同语言的协程式并发支持对比

语言/机制	抽象层次	调度主体	典型定位	特点概括
C++20 协程	偏底层	库/运行时决定	构建高性能异步框架	编译器提供状态机与协程帧机制，灵活但复杂 [7,8]
Go goroutine	中高层	Go runtime	高并发服务端工程	轻量、栈可增长、语言和运行时一体化 [4-6]
Python <code>asyncio</code>	高层	事件循环	异步 I/O、网络程序	<code>async/await</code> 直观，适合 I/O 绑定任务 [1,2]
Java 虚拟线程	中高层	JVM 运行时	高吞吐并发应用	保留线程式写法，降低高并发任务成本 [3]

从表中可以看出，几种语言并不是在“同一个层级”竞争。C++20 更像提供底层积木；Go 提供语言与运行时一体化的高并发模型；Python 强调事件循环与异步 I/O；Java 则以轻量线程的方式兼顾传统写法与高并发需求。

5.6 协程在工程实践中的应用与边界

从工程经验看，协程最适合下列任务：请求转发、连接管理、数据库访问、爬虫与异步采集、微服务网关、聊天或推流服务等。它们的共同点是：**并发数量大、单任务计算轻、等待时间长**。

但协程的边界同样需要明确。若任务主要是矩阵运算、图像处理、深度学习推理中的重计算环节，那么系统瓶颈通常不在“等待”，而在“算”。此时协程不会自动把计算变快，真正有效的优化手段更可能是多线程并行、SIMD、GPU 或专用加速库。

5.7 本章小结

本章比较了 C++20、Go、Python 与 Java 的协程式并发支持。它们的共同目标是降低等待型任务的管理成本，但抽象层次和运行时责任分配并不相同。理解这些差异，是后续分析实验结果与工程选型的重要前提。

6 实验设计与结果分析

6.1 实验目标

本文实验部分围绕五个问题展开：

- (1) 在大量模拟 I/O 等待任务下，协程相对于线程池是否有明显优势？
- (2) 在更接近真实网络编程的本地 HTTP 场景中，这种优势是否仍然存在？
- (3) 在 CPU 密集型任务中，协程是否仍会带来性能收益？
- (4) 当并发上限继续提高时，Python 中线程池与 `asyncio` 的吞吐、内存与尾延迟会如何变化？
- (5) Java 21 的虚拟线程在类似等待型任务中，相对于传统平台线程是否表现出更好的伸缩性？

对应地，本文设计了五组实验：Sleep I/O、Local HTTP、CPU-bound、Python scaling 和 Java scaling。前三组用于验证协程在等待型任务中的价值与边界；后两组用于观察系统在更高并发区间的扩展趋势，并补充跨语言视角。

6.2 实验环境与评价指标

本实验在 Windows 11 64 位操作系统上完成，处理器为 AMD Ryzen 7 7840H with Radeon 780M Graphics，具有 8 个物理核心和 16 个逻辑线程，内存为 15.19 GB。Python 实验使用 Python 3.12.7 作为实现语言，主要依赖库包括 `aiohttp` 3.13.5、`requests` 2.34.2、`psutil` 7.2.2、`pandas` 3.0.3 和 `matplotlib` 3.10.9。Java 扩展实验使用 Temurin OpenJDK 21.0.11，在同一台主机上完成。实验过程中采用本地回环地址构造 HTTP 服务，尽量减少外部网络波动对测试结果的影响；所有实验均重复执行 3 次，并对总耗时、吞吐量和峰值内存占用取平均值进行分析。

表 3: 实验环境配置

项目	配置
操作系统	Windows 11 64 位 (10.0.26200)
处理器	AMD Ryzen 7 7840H with Radeon 780M Graphics
物理核心数	8
逻辑线程数	16
内存容量	15.19 GB
Python 版本	Python 3.12.7
Java 版本	Temurin OpenJDK 21.0.11
Python 主要依赖	aiohttp 3.13.5, requests 2.34.2, psutil 7.2.2, pandas 3.0.3, matplotlib 3.10.9

本文使用的核心评价指标包括:

- **总耗时 (wall_time_s):** 完成全部任务所需的实际时间;
- **吞吐量 (QPS 或 throughput):** 单位时间内完成的任务数量;
- **峰值内存:** Python 中使用进程 RSS 峰值, Java 中使用已用堆内存峰值;
- **P95 延迟:** 扩展实验中统计的高分位延迟, 用于观察高并发区间的尾部行为。

需要特别说明的是, Python 与 Java 的内存口径并不相同: 前者更接近操作系统进程视角, 后者受 JVM 堆管理与 GC 行为影响更明显。因此, 跨语言内存图更适合用来观察**趋势**, 而不宜把绝对数值作直接横向比较。

6.3 实验一: 模拟 I/O 密集型任务下协程与多线程对比

实验一使用两种实现:

- 协程模型: `asyncio + asyncio.sleep`;
- 多线程模型: `ThreadPoolExecutor + time.sleep`。

任务规模取 {100, 1000, 5000, 10000}, 单任务等待时间取 {10 ms, 50 ms, 100 ms}, 线程池最大工作线程数设为 300; 每组参数重复 3 次并取平均值。这一实验的目的, 是尽量抽离真实网络栈带来的额外影响, 专门比较“等待型任务调度”的纯粹成本。

6.4 实验二: 真实网络 I/O 场景下协程与多线程对比

实验二在本机回环地址上搭建本地 HTTP 服务端, 服务端接收到请求后先等待指定时间, 再返回简单 JSON 响应。客户端分别采用:

- 协程模型: `aiohttp + asyncio`;
- 多线程模型: `requests + ThreadPoolExecutor`。

为了提高对比的公平性，实验中对两侧并发参数进行了显式限制：线程池并发数设置为固定上限，协程实现也使用 `TCPConnector` 与信号量对并发连接数进行控制。这一实验更加接近真实工程中的网络 I/O 负载。

6.5 实验三：CPU 密集型任务下协程与多线程对比

实验三构造一个纯计算循环，用累加 $\sum i^2$ 的方式模拟 CPU 密集型负载，并设置三档工作量 $work_n \in \{200000, 500000, 1000000\}$ 以及三档任务数 $\{10, 50, 100\}$ 。比较对象包括：

- 单线程顺序执行；
- 线程池执行；
- 基于 `asyncio.gather` 的协程式包装执行。

线程池最大工作线程数设为 8，每组参数重复 3 次并取平均值。该实验不是为了证明协程更快，而是为了检验协程在不适合场景中的表现，从而说明它的适用边界。由于本实验基于 Python 实现，这一对比的主要目的在于说明协程并不天然适合纯计算负载，而不是否定其他语言或原生并行环境中多线程对 CPU 密集任务的价值。

6.6 实验四：Python 高并发扩展实验

前三组实验主要回答“任务总量增加时谁更省成本”，而实验四则改变控制变量，转而观察“当并发上限继续提高时系统会发生什么”。该实验固定总任务数为 10000、单任务等待时间为 50 ms，并在每个任务开始前加入少量轻 CPU work。比较对象仍为：

- `ThreadPoolExecutor + time.sleep`；
- `asyncio + asyncio.sleep`。

并发度取 $\{100, 500, 1000, 2000, 3000, 5000\}$ ，每组参数重复 3 次，记录 QPS、P95 延迟与峰值 RSS。该实验的目标不是简单重复前三组结论，而是进一步观察高并发区间的扩展趋势、拐点与资源占用变化。

6.7 实验五：Java 21 平台线程与虚拟线程扩展实验

为弥补只在 Python 上做实证的局限，实验五在 Java 21 上构造与实验四近似的等待型负载：固定总任务数为 10000，每个任务执行少量轻 CPU work 后休眠 50 ms。比较对象为：

- `Executors.newFixedThreadPool(concurrency)` 构造的平台线程模型；
- `Executors.newVirtualThreadPerTaskExecutor()` 构造的虚拟线程模型。

并发度取 $\{100, 500, 1000, 2000, 3000\}$ ，每组参数重复 3 次，记录 QPS、P95 延迟和已用堆内存峰值。该实验并不声称虚拟线程与语言级协程等价，而是把它作为“轻量级并发执行单元”的另一种实现路线，与 Python 的事件循环模型形成横向对照。

6.8 基础机制验证实验结果

表 4 摘录了前三组实验中最具有代表性的结果。其中，Sleep I/O 选取任务数 10000，Local HTTP 选取请求数 3000，CPU-bound 选取任务数 100。对于前两组实验，基线模型为线程池；对于 CPU-bound，基线模型为单线程顺序执行。

表 4: 协程与多线程基础实验关键结果

实验类型	场景	任务数	协程耗时/s	基线耗时/s	倍率
Sleep I/O	10 ms	10000	0.125365	0.473125	3.774×
Sleep I/O	50 ms	10000	0.149345	1.764390	11.814×
Sleep I/O	100 ms	10000	0.191079	3.451140	18.061×
Local HTTP	10 ms	3000	0.364061	2.084250	5.725×
Local HTTP	50 ms	3000	1.082340	2.166350	2.002×
Local HTTP	100 ms	3000	1.872810	2.451250	1.309×
CPU-bound	$work_n = 200000$	100	1.054860	1.030810	1.023
CPU-bound	$work_n = 500000$	100	2.617340	2.654350	0.986
CPU-bound	$work_n = 1000000$	100	5.184820	5.226730	0.992

从表 4 可以看出，在 I/O 密集型场景下，协程相较于线程池具有明显优势，并且随着任务规模增大、等待时间增加，这种优势进一步放大。例如，在 Sleep I/O 场景中，当任务数为 10000、单任务等待时间为 100ms 时，协程实现的总耗时仅为 0.191079s，而线程池实现达到 3.451140s，前者相对于后者取得了约 18.06 倍的加速效果。在本地 HTTP 场景下，协程同样保持稳定领先，在 3000 个请求、10ms 延迟条件下，性能提升约为 5.73 倍。相比之下，在 CPU 密集型场景中，协程与单线程、线程池三者的耗时差异很小，性能比值基本接近 1，说明协程的核心价值在于高效管理大量等待中的任务，而不在于提升纯计算任务本身的执行速度。

下面分别给出三组基础实验的图形结果。

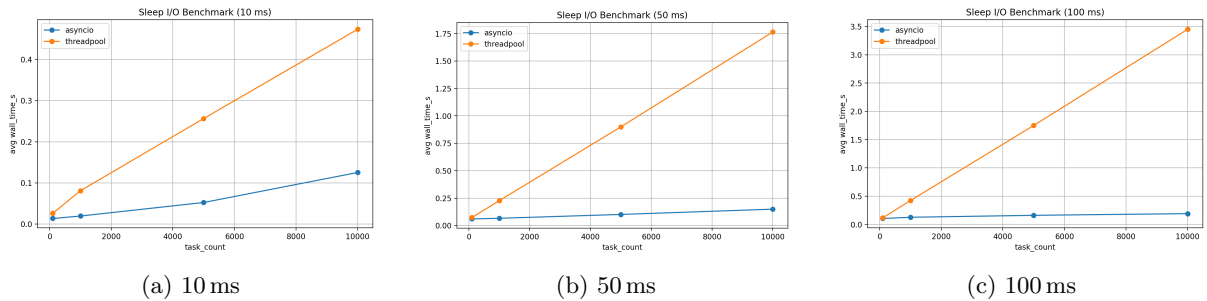


图 3: Sleep I/O 实验结果

图 3 显示，在模拟等待型任务中，`asyncio` 的总耗时随任务数增长得更慢，尤其在大规模任务和较长等待时间下优势非常明显。这说明协程降低了海量等待任务的调度与管理成本，是本文最核心的实验结论之一。

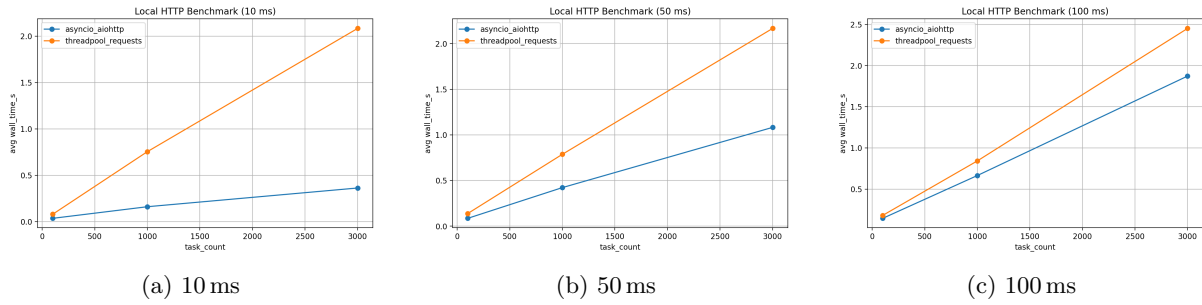


图 4: Local HTTP 实验结果

图 4 表明，在更接近真实网络编程的场景下，协程仍然保持稳定领先。与 Sleep I/O 相比，这里的优势有所收敛，说明当单请求等待时间继续拉长时，协议处理、HTTP 解析和连接管理开销在总耗时中所占比例会下降；但总体趋势仍然一致：在大量 I/O 请求下，协程实现更容易获得更高吞吐与更低总耗时。

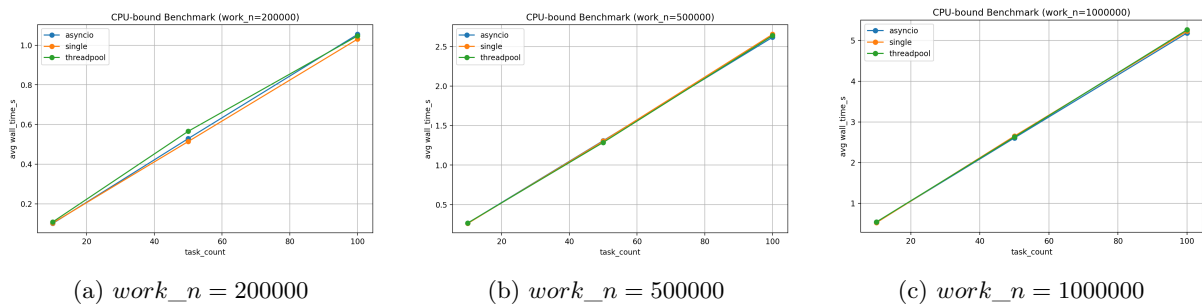


图 5: CPU-bound 实验结果

图 5 则给出了相反方向的证据：在纯 CPU 密集型任务中，协程、线程池和单线程顺序执行之间没有稳定而显著的差异。这说明协程并不会自动提升计算密集型程序的执行速度；当瓶颈来自计算本身时，真正重要的是并行计算资源和底层优化，而不是协程式控制流。

6.9 Python 高并发扩展实验结果

实验四的核心目的，是观察高并发上限继续推高时，`asyncio` 与线程池的吞吐和内存曲线如何变化。表 5 给出了 Python 扩展实验的摘要结果。

表 5: Python 高并发扩展实验摘要

并发度	asyncio QPS	threadpool QPS	asyncio P95/ms	threadpool P95/ms	内存趋势
100	1557.21	1943.03	6081.53	4748.86	95.06 vs 103.98
500	4845.76	4197.30	1970.30	1760.90	92.21 vs 133.99
1000	4709.96	4000.93	2030.73	1380.60	92.43 vs 171.52
2000	4615.00	3555.12	2080.72	632.86	94.00 vs 246.55
3000	4369.22	3222.36	2212.62	53.35	94.67 vs 312.98
5000	4343.15	3192.38	2238.24	53.10	95.21 vs 312.58

从 QPS 角度看，线程池在并发 100 时略高于 `asyncio`，但从 500 开始 `asyncio` 明显反超，并在 1000 之后持续保持更高吞吐；线程池则在并发继续提高后出现较明显回落。图 6 直观展示了这种现象。

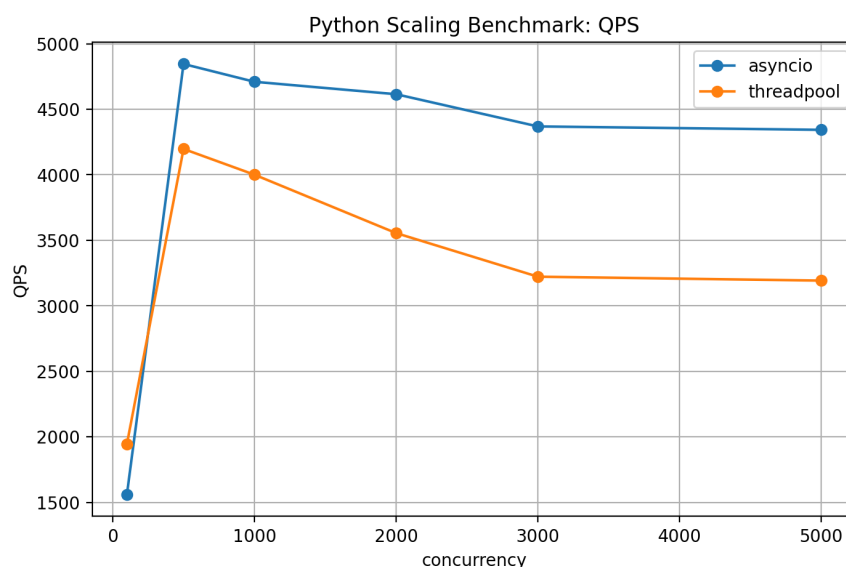


图 6: Python 高并发扩展实验: QPS

这表明，低并发区间里，事件循环和批量任务创建本身也有固定成本，因此 `asyncio` 不一定先天占优；但进入中高并发区间后，线程池排队、线程资源和切换成本逐渐主导开销，而 `asyncio` 更擅长管理大量等待型任务。不过，`asyncio` 的曲线也不是无限上升，在 3000 和 5000 并发时出现轻微平台和回落，这说明事件循环、任务创建和运行时开销在极高并发下同样会形成约束。

从内存角度看，`asyncio` 在整个并发区间内都保持在 90 MB 左右，而线程池随并发提高上升

到 300 MB 以上，如图 7 所示。

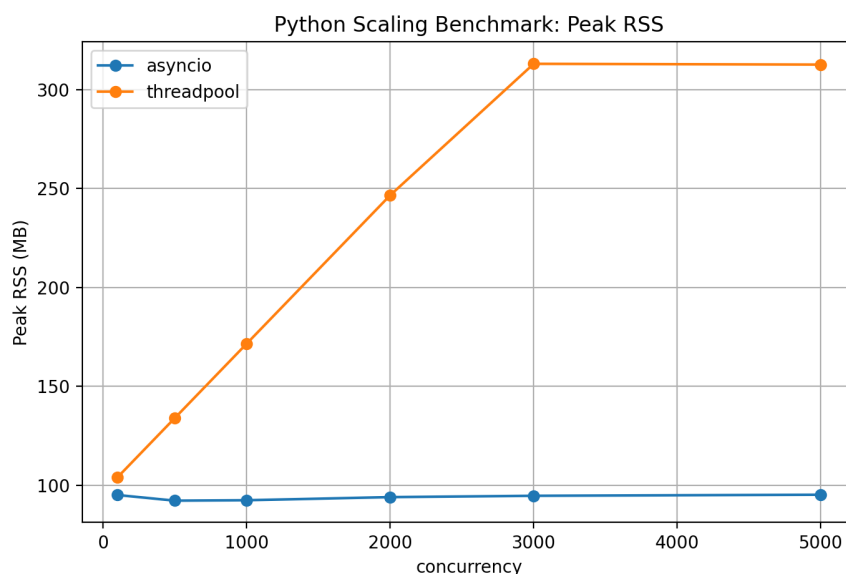


图 7: Python 高并发扩展实验：峰值 RSS

这条内存曲线非常符合协程的设计目标：在等待型任务规模扩大时，`asyncio` 主要增加的是任务对象和状态记录，而线程池需要承受更多线程资源和相关管理成本。

P95 延迟方面，图 8 也给出了结果，但其解释需要更谨慎。

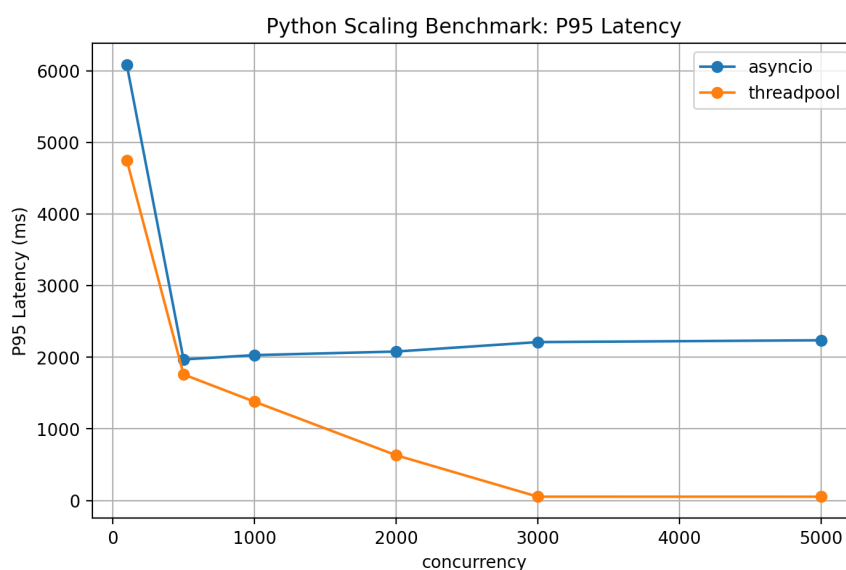


图 8: Python 高并发扩展实验：P95 延迟

由于该实验采用“固定总任务数、循环提交任务”的方式，任务在被调度之前已经存在提交顺序差异，因此高分位延迟除了反映运行时排队，还会受到任务提交时刻和统一起点定义的影响。换言之，这张图可以作为辅助观察，不宜像 QPS 和内存曲线那样被当作最强证据。因此，本文对 Python 扩展实验的主判断，仍然以 **QPS** 和 **峰值 RSS** 为主。

6.10 Java 21 平台线程与虚拟线程扩展实验结果

实验五进一步从 Java 21 的角度验证轻量级并发执行单元的扩展趋势。表 6 给出了平台线程与虚拟线程在不同并发度下的摘要结果。

表 6: Java 21 平台线程与虚拟线程扩展实验摘要

并发度	platform QPS	virtual QPS	platform P95/ms	virtual P95/ms	已用堆内存趋势
100	1604.64	1604.15	5914.67	5911.01	43.09 vs 53.22
500	7891.05	8023.56	1180.09	1181.45	75.22 vs 89.22
1000	14886.55	16048.01	606.66	621.04	95.22 vs 103.67
2000	23862.43	32266.81	287.00	306.56	119.67 vs 95.06
3000	24859.48	40798.09	184.50	242.14	106.39 vs 77.52

图 9 显示，平台线程与虚拟线程在并发 100 和 500 时差距很小，但从 1000 开始虚拟线程逐渐领先，到 2000 和 3000 时领先幅度进一步扩大。

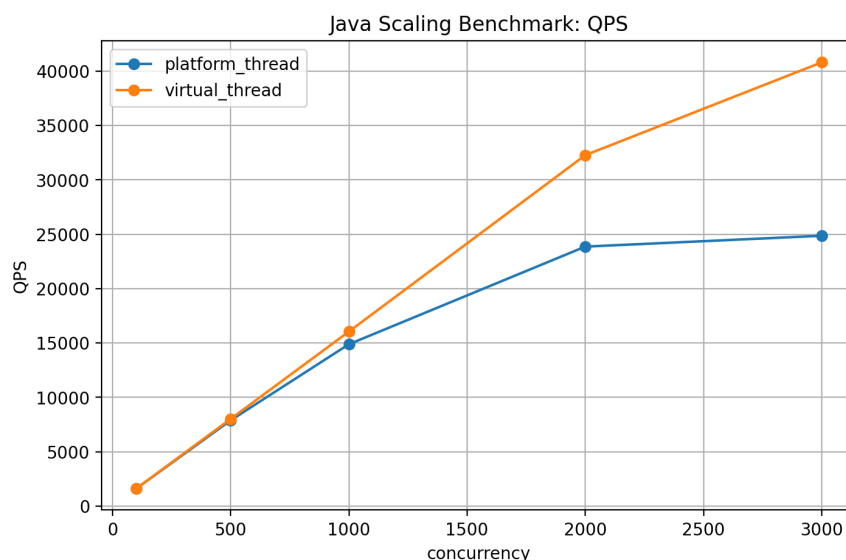


图 9: Java 21 扩展实验: QPS

这一结果与 JEP 444 想要解决的问题高度一致：虚拟线程并不是为了让单个任务更快，而是为了让 thread-per-request 风格在高并发等待型负载下仍具有伸缩性。当大量任务主要在等待而不是计算时，虚拟线程的调度成本更低，更容易持续扩大吞吐。

Java 的内存曲线见图 10。从图上看，虚拟线程在高并发区间的已用堆内存总体低于平台线程，但曲线也出现了一定波动。

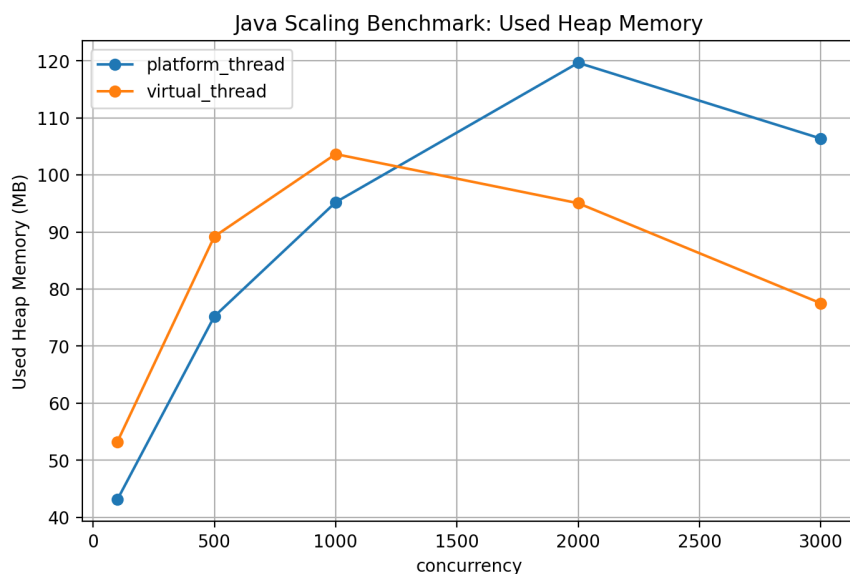


图 10: Java 21 扩展实验：已用堆内存

这种波动并不说明数据无效，而是提示我们：Java 的“已用堆内存”与 Python 的“进程 RSS”并不是同一种指标。前者会明显受到 GC 时机、堆扩容与回收策略影响，因此它更适合用于观察同一语言内部的趋势，而不宜与 Python 的 RSS 绝对值直接横向比较。也正因如此，本文对 Java 扩展实验的主结论同样优先依赖 QPS 曲线，而将内存图作为辅助证据。

Java 的 P95 延迟结果见图 11。

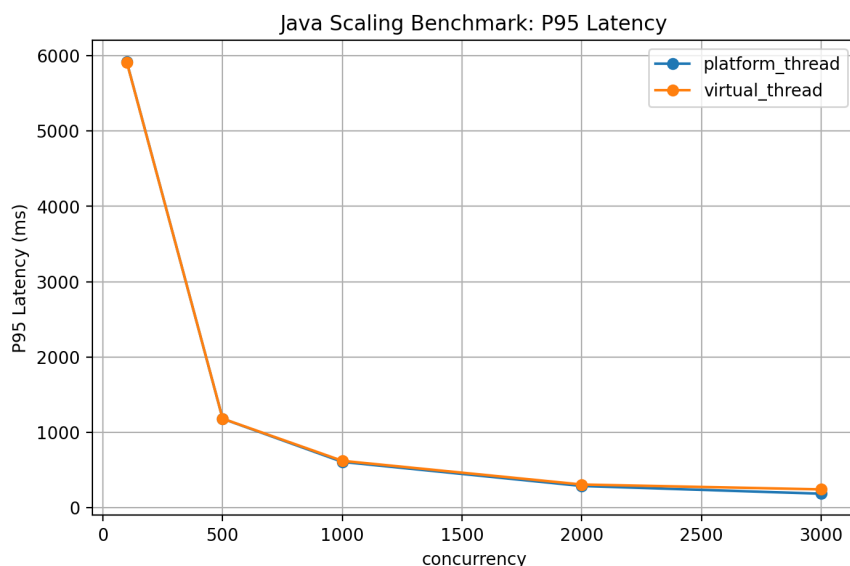


图 11: Java 21 扩展实验：P95 延迟

总体上，平台线程与虚拟线程的 P95 都随并发提高而下降，这与固定总任务数、提高并发度后分批执行轮数减少的实验设计是相符的。虚拟线程在 P95 上并未像 QPS 那样拉开特别大差距，这也提醒我们：不同指标反映的是系统的不同侧面，轻量级并发模型的最显著收益往往首先体现在吞吐扩展性与资源成本上。

6.11 综合讨论

把五组实验放在一起看，可以得到以下几条更完整的认识。

第一，前三组实验建立了本文最核心的机制判断：在等待型任务中，协程式并发相对线程池有明显优势；在 CPU 密集型任务中，这种优势并不成立。也就是说，协程的价值主要来自“高效管理等待”，而不是“自动加速计算”。

第二，Python 扩展实验进一步说明，这种优势并不意味着协程可以无限扩展。随着并发上限继续提高，`asyncio` 也会出现平台和波动，只是相对于线程池，它更晚触碰到明显的资源和调度瓶颈。

第三，Java 21 扩展实验补充了跨语言视角：即使不采用事件循环式协程，而是保留线程式写法，轻量级线程模型同样能够在等待型任务中表现出更好的吞吐扩展能力。这说明“轻量级并发执行单元”这一设计思想并不局限于单一语言范式。

第四，这些实验也提醒我们，评价并发模型时应当区分不同层面的指标。总耗时和 QPS 更适合回答“系统整体效率如何”，内存更适合回答“资源成本如何”，而 P95 这类高分位指标则需要结合实验方法谨慎解释。若实验的任务提交方式本身会影响延迟统计，那么高分位延迟就更适合作为辅助观察而非唯一证据。

6.12 本章小结

本章通过五组实验得出了较为清晰的结论：在模拟等待和真实网络 I/O 场景中，协程相比线程池具有明显优势，且任务规模越大、等待越长，优势越容易被放大；在 CPU 密集型场景中，协程并没有天然性能收益；在更高并发扩展实验中，Python 的 `asyncio` 和 Java 21 的虚拟线程都展示了更好的吞吐扩展性与更低或更平稳的资源成本。这些结果共同支持本文的核心判断：轻量级并发模型的价值，主要在于以更低成本组织大量等待中的任务，而不是加速计算本身。

7 结论与展望

7.1 全文总结

本文围绕“为什么并发模型会从线程演进到协程式并发”这一问题展开分析。首先，本文说明了进程、线程、事件驱动与协程之间的演进逻辑：线程虽然降低了进程的代价，但在高并发 I/O 场景下仍有较大的资源浪费；事件驱动提高了效率，却牺牲了代码直观性；协程则试图在二者之间取得平衡。随后，本文从概念层面对协程的定义、分类与适用场景进行了归纳，并从编译器、运行时和操作系统三层视角解释了其底层实现机制。之后，本文比较了 C++20、Go、Python 与 Java 四种代表性语言或平台对协程式并发的支持方式，指出它们虽然目标相近，但抽象层次与实现职责并不相同。最后，本文通过五组实验验证了协程在 I/O 密集型任务中的优势、其在 CPU 密集型任务中的边界，以及轻量级并发模型在更高并发区间的扩展趋势。

7.2 主要结论

综合全文分析，可以归纳出以下几点结论：

- (1) 协程的出现并不是为了“替代线程的一切作用”，而是为了在高并发 I/O 场景下，以更低成本组织大量等待中的任务。
- (2) 协程的底层实现依赖编译器、运行时与操作系统三者协同：编译器负责状态机与协程帧，运行时负责调度与唤醒，操作系统负责线程与 I/O 事件。

- (3) 不同语言对协程式并发的支持层次不同：C++20 偏底层机制，Go 偏语言与运行时一体化，Python 偏事件循环模型，Java 虚拟线程则试图保留传统线程式写法的可读性。
- (4) 基础机制验证实验表明，在 I/O 密集型场景中协程具有显著优势；在 CPU 密集型场景中，这种优势并不成立。
- (5) 扩展实验表明，轻量级并发模型在更高并发区间往往具有更好的吞吐扩展性与资源利用率，但这种优势也不是无限的；系统最终仍会受到运行时、调度器和底层资源约束。

归根结底，协程的意义不在于消灭线程，而在于把海量等待型任务从“线程一一映射”的模式中解放出来。

7.3 未来发展趋势与进一步研究方向

从未来趋势看，轻量级并发抽象仍会持续发展。一个方向是语言层与运行时层进一步协同，例如更完善的结构化并发、取消传播、任务组管理和可观测性支持；另一个方向是协程与系统底层能力之间的结合更紧密，例如对 I/O 多路复用、用户态网络栈以及异步文件系统的更充分利用。

进一步研究时，可以从以下几个方面深化：

- 在 C++ 中使用真正的协程库或网络库（如基于 `co_await` 的异步框架）进行对照实验；
- 在 Go 平台复现实验，比较 goroutine 与 Python 事件循环、Java 虚拟线程之间的运行时差异；
- 改进高并发扩展实验中的延迟统计方法，例如使用统一批次起点、更精确的到达过程建模与服务时间建模，以得到更稳定可信的尾延迟结论；
- 进一步考察取消、异常传播、背压与任务泄漏等工程问题，而不仅限于总耗时、QPS 和内存等指标。

A 附录：实验代码

A.1 GitHub 项目链接

本文实验代码统一整理于 GitHub 仓库：

<https://github.com/ZR-1N/ot-coroutine-study>

A.2 多线程实验代码

- 多线程实现主要位于项目目录 `experiments/io_sleep/run_sleep_benchmark.py` 中。
- 多线程实现还位于
`experiments/http_local/run_http_benchmark.py`。
`experiments/cpu_bound/run_cpu_benchmark.py`。
- 对应的多线程部分分别基于 `ThreadPoolExecutor` 与 `requests` 等库实现。
- 扩展实验中的线程池代码位于 `experiments/python_scaling/run_scaling_benchmark.py` 中。
- 扩展实验中的线程池代码还位于
`experiments/java_scaling/src/VirtualThreadScalingBenchmark.java`。

A.3 协程与虚拟线程实验代码

协程实现位于与上述文件相同的脚本中，其中 Sleep I/O 实验使用 `asyncio.sleep`，HTTP 实验使用 `aiohttp + asyncio`，CPU-bound 实验则用 `asyncio.gather` 对计算任务进行封装。Java 扩展实验则同时实现了 `newFixedThreadPool` 和 `newVirtualThreadPerTaskExecutor` 两种执行模式，用于比较平台线程与虚拟线程的差异。

A.4 数据处理与绘图代码

- 实验数据汇总与绘图代码主要位于 `experiments/analysis/plot_results.py` 中。
- 实验数据汇总与绘图代码还位于 `experiments/analysis/plot_scaling_results.py` 中。
- 实验数据汇总与绘图代码还位于 `experiments/analysis/make_summary_tables.py` 中。
- 实验数据汇总与绘图代码还位于 `experiments/analysis/collect_env_info.py` 中。
- 这些脚本用于自动生成结果图、摘要表以及实验环境信息。
- 该部分保证了实验结果可以重复生成和统一整理。

B 附录：AI 辅助使用说明

B.1 AI 辅助内容

本报告在撰写过程中使用了大语言模型辅助完成以下工作：

- 帮助梳理协程调研报告的整体结构；
- 协助设计实验方案、代码组织方式与结果表格；
- 对部分表述进行润色，并根据官方文档整理参考文献格式；
- 协助生成 LaTeX 图示、表格骨架和图片插入结构。

B.2 人工核验与修改说明

所有实验均由本人在本地环境中实际运行，结果图表与摘要表由本人生成并检查。报告中的章节组织、实验取舍、结论归纳与最终文字内容均经过本人修改与确认。对于关键结论，本文尽量结合官方文档与实验结果进行双重支撑，以避免仅依赖生成式工具给出未经核验的表述。

参考文献

- [1] Python Software Foundation. `asyncio` — asynchronous i/o, 2026. Python documentation, accessed 2026-05-30.
- [2] Python Software Foundation. Coroutines and tasks, 2026. Python documentation, accessed 2026-05-30.
- [3] Ron Pressler and Alan Bateman. JEP 444: Virtual threads, 2025. OpenJDK JEP, updated 2025-10-30, accessed 2026-05-30.

- [4] The Go Authors. Effective go, 2026. Go documentation, accessed 2026-05-30.
- [5] The Go Authors. Frequently asked questions (faq), 2026. Go documentation, accessed 2026-05-30.
- [6] The Go Authors. The go programming language: runtime/hacking, 2026. Go runtime documentation, accessed 2026-05-30.
- [7] Microsoft. /await (enable coroutine support), 2025. Microsoft Learn, accessed 2026-05-30.
- [8] LLVM Project. Debugging c++ coroutines, 2026. Clang documentation, accessed 2026-05-30.